

Sorting Algorithms

Sorting algorithms arrange the elements of a list in a specific order, typically ascending or descending. Mastery of their time complexities, stability, and underlying paradigms is fundamental for analyzing efficiency in algorithm design exams.

Definition and Fundamentals

- Sorting rearranges an array or list of items according to a comparison operator.
- A sorting algorithm is stable if it preserves the relative order of items with identical keys.
- An in-place sorting algorithm requires a constant amount of extra memory space $O(1)$ beyond the input items.
- Comparison-based sorts rely on evaluating pairs of elements to determine their relative order.

Comparison Lower Bound Theory

- Any comparison-based sorting algorithm can be modeled as a decision tree with $n!$ leaves.
- The height h of a binary decision tree with L leaves satisfies $h \geq \log_2(L)$.
- This establishes the theoretical worst-case lower bound for comparison sorts as $\Omega(n \log n)$.
- Non-comparison sorts like Counting Sort bypass this limit by exploiting direct access to key values.

Formula: $h \geq \log_2(n!) = \Theta(n \log n)$

Divide and Conquer: Merge Sort

- Merge Sort recursively splits the array into two halves until subarrays contain a single element.
- It then merges the sorted subarrays in linear time to produce the final sorted output.
- The recurrence relation reflects dividing the problem in half and linear merging work.
- Merge Sort is stable and guarantees worst-case time complexity of $O(n \log n)$, but requires $O(n)$ auxiliary space.

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

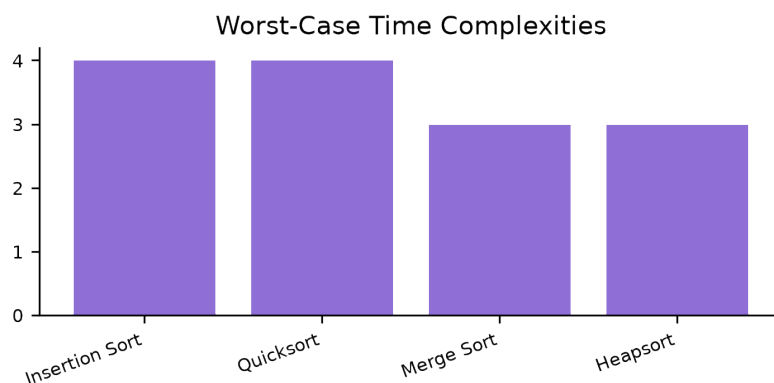
Merge Sort Execution Flow



Comparison of Sorting Complexities

- Quadratic sorts like Insertion and Selection perform well on small or nearly sorted datasets.
- Merge Sort and Heapsort guarantee $O(n \log n)$ worst-case time but differ in space complexity.
- Quicksort offers superior average-case performance with $O(n \log n)$ but degrades to $O(n^2)$ in the worst case without good pivot selection.
- Linear time sorts achieve $O(n + k)$ by avoiding direct pairwise comparisons entirely.

$$T_{worst}(Quicksort) = \Theta(n^2)$$



Applications

- Sorting serves as a critical preprocessing step for binary search, greedy algorithms, and two-pointer techniques.
- Database query optimization uses sorted data to efficiently perform joins and group-by operations.
- Information retrieval systems sort search engine results by relevance score.
- Computer graphics utilize Z-buffering and sorting primitives to handle visible surface determination.

Key Terms

Stability	Property of a sorting algorithm that maintains the relative input order of identical elements.
In-place	An algorithm that uses constant extra memory auxiliary space $O(1)$ beyond the input storage.
Decision Tree	A binary tree model used to prove the lower bound of comparison-based sorting algorithms.
Partitioning	The core subroutine of Quicksort that arranges elements around a chosen pivot value.

Recurrence Relation	An equation that defines a sequence recursively, commonly solved using the Master Theorem for divide-and-conquer sorts.
Counting Sort	A non-comparison linear time sorting algorithm that counts the frequency of each distinct key value.

Exam Tips

- ★ Always state whether a sorting algorithm is stable when comparing performance metrics in exam questions.
- ★ Be prepared to use the Master Theorem to solve recurrence relations for divide-and-conquer sorting algorithms like Merge Sort.
- ★ Remember that Quicksort's worst-case $O(n^2)$ occurs when the pivot selection consistently picks the minimum or maximum element.
- ★ Know the distinction between comparison-based lower bounds and linear-time non-comparison sorting algorithms.